



Politechnika
Wrocławska

PROJEKT ZESPOŁOWY

Autor:

Kasper Radom 264023
Mikołaj Wiora 266895
Aleksander Pajorski 264203
Dawid Boratyński 268094

Grupa, termin:

gr 2, Piątek,
7:30 – 11:00

Data oddania:

23.01.2026 r

Temat projektu:

Zespół programistyczny Agentów AI

Ocena

Spis treści

1. Wprowadzenie	3
1.1. Cel projektu.....	3
1.2. Zakres i ograniczenia projektu.....	3
2. Przegląd literatury	6
2.1. Multi-Agent-based Code Generation with Iterative Testing and Optimisation	6
2.2. LLM-Based Multi-Agent Systems for Software Engineering	6
2.3. Optimizing LLM-Based Multi-Agent System with Textual Feedback.....	6
2.4. Towards Building Multi-Agent Systems with Heterogeneous LLMs X-MAS.....	6
3. Architektura	8
3.1. Architektura warstwowa	8
3.2. Architektura Event-Driven.....	8
3.3. Rola komponentów	9
3.4. Schemat komunikacji.....	10
4. Wdrożenie	11
4.1. Wybór dostawcy LLM.....	11
4.2. Organizacja kluczy API	12
4.3. Communication Service.....	14
4.4. Implementacja Agentów	14
4.5. Test Machine.....	15
4.5.1. Technologia i Implementacja	15
4.5.2. Logika Operacyjna i Kryteria Decyzyjne.....	15
4.6. Prompting.....	16
4.7. Frontend	16
5. Testy i Walidacja Systemu.....	18
5.1. Scenariusz 1: Aplikacja Konsolowa (Flappy Bird).....	18
5.2. Scenariusz 2: Aplikacje GUI (Pygame).....	18
5.2.1. Kalkulator	18
5.2.2. Gra w życie (Game of Life).....	19
5.2.3. Snake	19
6. Wnioski	20
7. Bibliografia	22

1. Wprowadzenie

Projekt Zespół Programistyczny AI zakłada stworzenie zautomatyzowanego zespołu programistycznego opartego na technologii sztucznej inteligencji. Zespół ten będzie składał się z kilku wyspecjalizowanych modeli AI, z których każdy będzie pełnił określoną rolę w procesie tworzenia oprogramowania, podobnie jak w rzeczywistym zespole deweloperskim.

Głównym zadaniem jest podział zadań i umożliwienie współpracy między różnymi agentami AI czyli modelami językowymi typu LLM, które wspólnie będą analizować wymagania, projektować rozwiązania, pisać kod, testować go oraz oceniać jego jakość.

System będzie w oparciu o instrukcje użytkownika symulował proces wytwarzania oprogramowania od etapu analizy wymagań po końcową walidację jakości, z naciskiem na komunikację i koordynację między poszczególnymi agentami, dając przy tym użytkownikowi możliwość śledzenia pracy zespołu.

1.1. Cel projektu

Głównym celem projektu jest opracowanie prototypu wirtualnego zespołu programistycznego AI, który potrafi współpracować i wymieniać się informacjami w celu stworzenia prostego rozwiązania IT tj. mała aplikacja webowa lub mobilna. Głównymi celami są:

- Stworzenie mechanizmu komunikacji i koordynacji między agentami AI.
- Opracowanie interfejsu, który umożliwi obserwację przebiegu i postępu prac zespołu AI.
- Zbadanie skuteczności i potencjału automatyzacji procesów wytwarzania oprogramowania z wykorzystaniem modeli językowych.
- Ocena jakości kodu generowanego przez zespół AI w porównaniu do kodu pisanego przez człowieka.

1.2. Zakres i ograniczenia projektu

W ramach precyzyjnego wyznaczenia zakresu realizacji projektu, zdefiniowano funkcjonalny zakres systemu oraz przedstawiono techniczne i budżetowe bariery wpływające na ostateczny kształt efektu końcowego. Głównymi ograniczeniami są wymagania stawiane przez rozwiązania strukturalne, wymagane ze względu na kluczowe założenia dotyczące architektury i symulacji procesów deweloperskich. Są to między innymi limity API czy brak trwałej pamięci kontekstowej. Uzupełnieniem tej charakterystyki jest analiza punktów funkcyjnych (FPA), wykonana na wczesnym etapie projektu, służąca do obiektywnego oszacowania złożoności projektowanego rozwiązania, przedstawiona w tabeli 1.

Projekt będzie realizowany zgodnie z założeniami i wymaganiami kursu ‘Projekt Zespołowy’ na Politechnice Wrocławskiej, korzystając z udostępnionych na Eportalu materiałów pomocniczych. [1]

Nazwa funkcji	Typ (EI/EO/EE/IL F/EIF)	DET (pola)	FTR/RET (pliki/reko rdy)	Złożoność (Niska/Średnia /Wysoka)	Ilość	Waga (auto)	Punkty (Ilość*Waga)
Stworzenie wizji projektu	EI			Wysoka	1	6	6
Stworzenie modelu współpracy	EI			Średnia	1	4	4
Raport wizji projektu	EO			Średnia	1	5	5
Przygotowanie Infrastruktury współpracy	EI			Średnia	1	4	4
Zaprojektowanie modelu danych	EI			Średnia	1	4	4
Zaprojektowanie architektury komunikacji	EI			Średnia	1	4	4
Raport architektury projektu	EO			Średnia	1	5	5
Wybór dostawcy LLM	EI			Średnia	1	4	4
Organizacja kluczy API	EI			Wysoka	1	6	6
Baza Communication Service	ILF			Wysoka	1	15	15
Integracja modelu Planera	EI			Wysoka	1	6	6
Integracja modeli Programistów	EI			Wysoka	1	6	6
Integracja modelu Reviewer	EI			Wysoka	1	6	6
Wdrożenie Test Machine	EI			Średnia	1	4	4
Raport wdrożeniowy	EO			Średnia	1	5	5
Podgląd statusu workflow agentów	EE			Średnia	1	4	4
Podgląd historii komunikacji	EE			Średnia	1	4	4
Interfejs użytkownika	EE			Wysoka	1	6	6
Analiza optymalizacyjna projektu	EO			Średnia	1	5	5
Wdrożenie poprawek	EI			Średnia	1	4	4
Testy akceptacyjne	EO			Niska	1	4	4
Dokumentacja i prezentacja	EO			Średnia	1	5	5
Raport końcowy	EO			Średnia	1	5	5

Suma UFP/NPF:	121
----------------------	------------

Tabela 1: Analiza złożoności projektu względem modelu UFP [1]

Zakres projektu:

- Stworzenie architektury systemu, który umożliwia współdziałanie kilku agentów AI o różnych rolach.
- Implementacja prostego mechanizmu zarządzania komunikacją między agentami.
- Integracja z modelem językowym w celu generowania, testowania oraz analizy kodu.
- Opracowanie prototypowego środowiska do obserwacji wyników działania zespołu

Ograniczenia projektu:

- Projekt ma charakter badawczo-prototypowy - nie jest przeznaczony do wdrożenia komercyjnego.
- Brak trwałej pamięci kontekstowej, brak pełnej świadomości stanu projektu modelu językowego.
- Brak rzeczywistej integracji z narzędziami CI/CD - symulacja pracy zespołu ograniczona do wymiany tekstowej i generowania kodu.
- Problem wysokiego kosztu i ograniczeń API modeli AI (limity tokenów, czas odpowiedzi itp).
- Brak fizycznego komponentu decyzyjnego - koordynator AI będzie działał według ustalonych reguł, nie autonomicznie.

2. Przegląd literatury

Współczesna inżynieria oprogramowania wspomagana sztuczną inteligencją przechodzi ewolucję od wykorzystania pojedynczych modeli językowych (LLM) w stronę złożonych systemów wieloagentowych (Multi-Agent Systems). Niniejszy rozdział przedstawia analizę wybranych architektur i metodologii implementacji podobnych rozwiązań, które stanowią teoretyczną podbudowę dla realizowanego projektu.

2.1. AgentCoder: Multi-Agent-based Code Generation with Iterative Testing and Optimisation

AgentCoder to framework, który rozwiązuje problem generowania kodu poprzez współpracę trzech wyspecjalizowanych agentów: programisty (generującego kod), projektanta testów (tworzącego przypadki testowe) oraz wykonawcy testów (uruchamiającego kod). Główną innowacją jest tutaj pętla sprzężenia zwrotnego jeśli testy nie przejdą, agent-programista otrzymuje wynik i poprawia kod, aż do skutku. System ten udowadnia, że oddzielenie procesu generowania testów od samego kodowania znacząco podnosi jakość końcowego oprogramowania w porównaniu do pojedynczego modelu LLM. [2]

2.2. LLM-Based Multi-Agent Systems for Software Engineering

Tego typu systemy (często omawiane w pracach przeglądowych lub implementowane jako ChatDev) traktują proces wytwarzania oprogramowania jako sekwencję zadań w wirtualnej firmie programistycznej. Agenty wcielają się w role takie jak CEO, Product Manager, CTO czy Programista i komunikują się ze sobą w ustalonym porządku (wodospadowym lub zwinnym), aby przekształcić abstrakcyjne wymaganie w konkretny produkt. Nacisk kładziony jest tutaj na strukturę komunikacji i "łańcuch myśli" rozproszony pomiędzy wieloma aktorami, co pozwala na dekompozycję skomplikowanych zadań na prostsze instrukcje. [3]

2.3. Optimizing LLM-Based Multi-Agent System with Textual Feedback

Podejście to koncentruje się na ulepszaniu działania systemu wieloagentowego nie poprzez zmianę wag modelu (co jest kosztowne i trudne), ale poprzez optymalizację samej treści komunikatów i promptów (prompt engineering) na podstawie informacji zwrotnej. System analizuje historię błędów i sukcesów w komunikacji tekstowej między agentami, a następnie dynamicznie modyfikuje instrukcje, aby agenty lepiej "rozumiały" swoje zadania w kolejnych krokach. Jest to forma uczenia się systemu w czasie rzeczywistym (in-context learning) bez konieczności kosztownego douczania modelu (fine-tuning). [4]

2.4. Towards Building Multi-Agent Systems with Heterogeneous LLMs (X-MAS)

X-MAS (eXtensible Multi-Agent System) to architektura promująca wykorzystanie heterogenicznych (różnorodnych) modeli językowych w ramach jednego systemu. Zamiast używać jednego, potężnego i drogiego modelu (np. GPT-4) do wszystkiego, system deleguje zadania do różnych modeli w zależności od trudności: proste zadania trafiają do mniejszych, tańszych modeli, a kluczowe decyzje do tych najbardziej zaawansowanych. Pozwala to na zachowanie wysokiej jakości przy drastycznym obniżeniu kosztów operacyjnych i czasu przetwarzania. [5]

3. Architektura

W świetle książki Richardsa projekt można opisać jako hybrydę kilku architektonicznych podejść, w zależności od tego, która część systemu zostanie przeanalizowana. Ostateczny werdykt jednak padł na hybrydę architektur warstwowej oraz event-driven, z naciskiem na tę drugą.

3.1. Architektura warstwowa

Zgodnie z wizją projektu, system ma wyraźne role i separację odpowiedzialności, co jest fundamentem wzorca layered architecture. Projekt posiada trójwarstwową strukturę: interfejs użytkownika, logikę koordynatora oraz warstwę wykonawczą składającą się z wyspecjalizowanych agentów AI.

- Warstwa prezentacji (UI) - odpowiada za interakcję z użytkownikiem i obserwację dialogów agentów.
- Warstwa koordynacyjna (AI Koordynator) - zarządza przepływem informacji, kolejką zadań, logiką procesu.
- Warstwa wykonawcza (AI Programista, Tester) - komponenty realizujące konkretne funkcje.
- Warstwa danych - formaty wiadomości, struktura JSON, dane wejściowe/wyjściowe dla agentów.

3.2. Architektura Event-Driven

Zgodnie z wizją projektu Koordynator rozdziela zadania i zbiera odpowiedzi, zadania są przekazywane między agentami w formie komunikatów JSON, z kolejne cykle pracy wyznacza kolejka zadań.

Idealnie pasuje więc to do Topologii Mediatora opisanej w książce Richardsa, ponieważ Koordynator orkiestruje proces od wymagań → kod → testy → analiza. System AI działa według wzorca Event-Driven Architecture w topologii Mediator. Zgodnie z opisem przedstawionym przez Martina Fowlera [6], jest to rodzaj architektury w której centralny Koordynator zleca zadania wyspecjalizowanym agentom i zbiera wyniki ich pracy. Nie wykonuje pracy, tylko wie, jaki krok ma nastąpić, komu przekazać zadanie oraz jak zbierać wyniki, identycznie jak mediator:

- nie pisze kodu, robi to Programista,
- nie testuje kodu, robi to Tester,
- sam tylko steruje przepływem procesu

Pełne rzutowanie elementów EDA na projekt zaprezentowano w tabeli 1.

Element EDA	Odpowiednik w projekcie
Event mediator	AI Koordynator
Event processors	AI Programista, AI Tester
Event channels	kolejka zadań
Event queues	przydzielenie zadania, kod wygenerowany, wyniki testów

Tabela 1: Rzutowanie architektury event-driven na projekt [6].

3.3. Rola komponentów

Planner

- Analizuje ogólny opis aplikacji otrzymany w prompcie
- Generuje listę klas potrzebnych do zaimplementowania systemu
- Określa strukturę i zależności między komponentami
- Wprowadza poprawki zapewniające większą spójność klas

Dev

- Implementuje klasę określoną przez Plannera

Rewiewer

- Otrzymuje kod do poprawy w przypadku błędów testowych

Tester

- Otrzymuje klasę od Developera, na podstawie której tworzy testy jednostkowe
- Nie uruchamia testów - tylko generuje kod testowy

Test Machine

- Uruchamia testy i aplikację
- Zbiera i zwraca logi z uruchomień
- W przypadku błędów: przesyła do Rewiewer informacje o błędach do naprawy
- Przy braku błędów: przekazuje do następnego etapu

Comunication Service

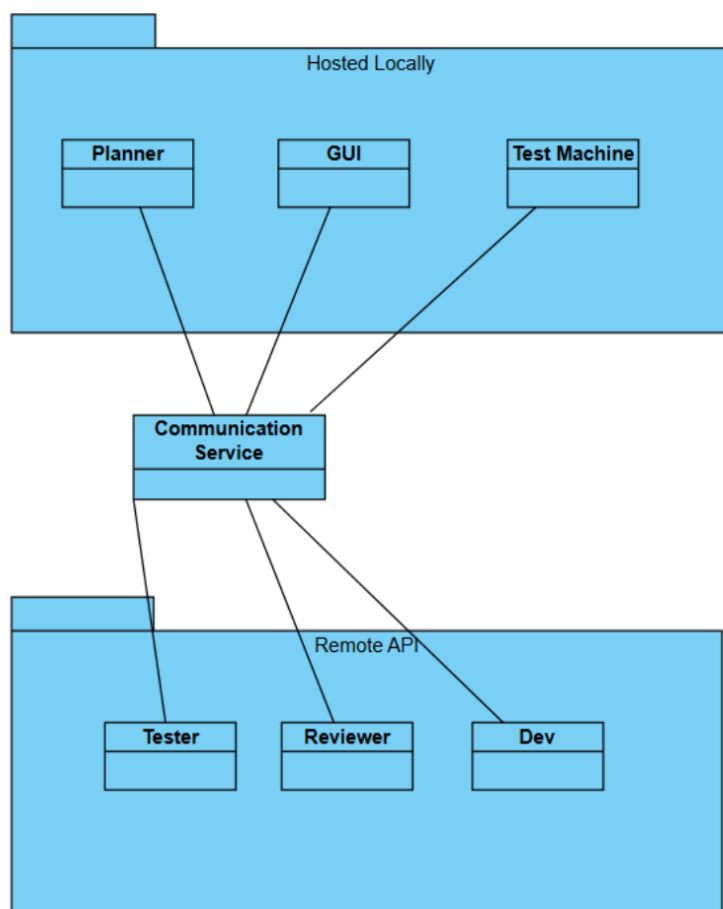
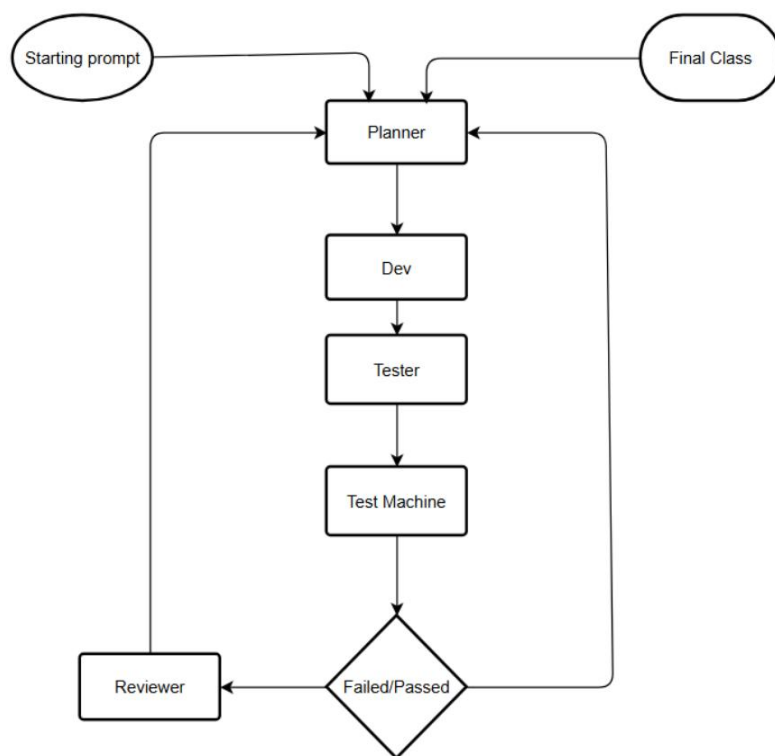
- Obsługuje całą komunikację pomiędzy agentami
- Zapisuje historię konwersacji między komponentami

GUI

- Pozwala na wprowadzenie opisu aplikacji przez użytkownika
- Prezentacja komunikacji między agentami
- Zwraca gotowe pliki programu

Szczegółowy model interakcji między elementami systemu przedstawiono na rysunku 1.

3.4. Schemat komunikacji



Rysunek 1: Workflow systemu. [opracowanie własne]

4. Wdrożenie

Pierwszym etapem wdrożenia był wybór odpowiedniego Dużego Modelu Językowego, który stanowi silnik inferencyjny dla wszystkich agentów. W dalszej kolejności skoncentrowano się na aspektach bezpieczeństwa i zarządzania dostępem (klucze API) oraz budowie centralnego węzła wymiany danych, jako serwisu komunikacyjnego. Druga faza wdrożenia obejmowała programowanie logiki poszczególnych ról: od strategicznego Planera, przez wykonawczych Programistów, aż po kontrolującego jakość Reviewera. Całość procesu zwieńczona została implementacją zautomatyzowanego środowiska testowego Test Machine, stanowiącego ostateczny filtr weryfikacyjny w potoku wytwarzania oprogramowania.

4.1. Wybór dostawcy LLM

W ramach projektu rozważano kilka grup dostawców LLM: Komercyjnych dostawców jak OpenAI, Anthropic, Google czy Mistral oraz modele open-weight jak choćby Llama 3 czy inne modele uruchamiane lokalnie.

Kluczowymi kryteriami wyboru były jakość generowanego kodu, koszt (TCO w trakcie trwania projektu), łatwość integracji z Pythonem oraz kwestie prywatności danych.

Ostatecznie zdecydowano się na Google AI Studio API jako dostawcę podstawowego, w ramach, którego dostępne są obecnie modele gemini-1.5-flash, gemini-1.5-pro, gemini-2.0. Wybór ten zapewnia:

- najwyższą jakość generacji i analizy kodu dla agentów Programisty i Reviewera,
- proste w użyciu API,
- wybór korzystny z punktu widzenia TCO, ze względu na darmowy dostęp do Gemini dla studentów.

Alternatywną opcję użycia modelu open-source Llama 3, uruchamianego lokalnie, odrzucono ze względu na dodatkową złożoność infrastrukturalną i ograniczony czas trwania projektu.

Parametry kluczowe z punktu widzenia projektu w zakresie wyboru dostawcy modeli AI, przedstawiono w tabeli 2.

Cecha	Google Gemini (AI Studio)	OpenAI (GPT-4o / mini)	Anthropic (Claude 3.5)
Darmowe API?	TAK (Free Tier)	NIE (Tylko próbne kredyty \$5 na start, potem płatne)	NIE (Trzeba wpłacić min. \$5 przedpłaty)
Okno kontekstowe	1 mln - 2 mln	128k	200k
Rate Limits (Darmowe)	15 RPM (Flash)	Zależne od wydanej kwoty (Tier 1 jest niski)	Zależne od wpłaty

Tabela 2: Kluczowe parametry doboru dostawcy usług AI.

4.2. Organizacja kluczy API

W prezentowanym rozwiązaniu zaimplementowano mechanizm rotacji kluczy API, umożliwiający komunikację z usługą Google AI Studio, zapewniającą dostęp do modeli z rodziny Gemini. Ze względów bezpieczeństwa oraz w celu łatwej konfiguracji, klucze nie są wpisane w kodzie na sztywno, lecz są pobierane dynamicznie z zewnętrznego pliku konfiguracyjnego ".env". System automatycznie wykrywa wszystkie dostępne w środowisku klucze, co pozwala na płynne przełączanie się między nimi w przypadku osiągnięcia limitów zapytań (Rate Limits).

Założenia:

- Klucze nie mogą znajdować się w kodzie źródłowym ani w systemie kontroli wersji.
- System musi przed uruchomieniem agentów sprawdzić, czy klucz jest poprawny i czy ma odpowiednie uprawnienia, aby nie "wywalić się" w połowie pracy Planera.

Inicjalizacja:

Communication Service → .env (Pobierz API_KEY)

.env → Communication Service (Zwraca klucz)

Communication Service (walidacja klucza)

Communication Service → Gemini SDK (konfiguracja po kluczu)

Gemini SDK → Communication Service (potwierdzenie autoryzacji sesji)

W celu optymalizacji wykorzystania dostępnych zasobów oraz minimalizacji ryzyka wystąpienia błędów związanych z limitami zapytań narzuconymi przez platformę Google AI

Studio, w systemie zaimplementowano mechanizm rotacji kluczy oparty na algorytmie Round Robin.

Klucze autoryzacyjne pobierane są dynamicznie z pliku konfiguracyjnego, co pozwala na elastyczne zarządzanie pulą dostępnych zasobów bez ingerencji w kod źródłowy. Zastosowanie metody Round Robin polega na sekwencyjnym i cyklicznym przydzielaniu kolejnych kluczy z listy do każdego nowego zapytania lub instancji modelu. Dzięki temu obciążenie jest równomiernie rozkładane na wszystkie zdefiniowane konta, co znacząco wydłuża czas ciągłej pracy agentów przed osiągnięciem limitów (tzw. *quota*).

Dokładny schemat zaimplementowanego rozwiązania algorytmu rotacyjnego przedstawiono na podstawie pseudokodu poniżej.

```
load_dotenv()                                //Inicjalizacja zmiennych środowiskowych

klucze = [                                    // Pula kluczy
os.getenv("API_KEY_1"),
os.getenv("API_KEY_2"),
os.getenv("API_KEY_3"),
os.getenv("API_KEY_4")
]

def daj_mi_model(): global licznik

obecny_klucz = klucze[licznik % 4] //Wybierz klucz w kolejności 0, 1, 2, 3, 0, 1...

licznik += 1

configure(api_key=obecny_klucz)
//Konfiguracja po n-tym kluczu return GenerativeModel('gemini-1.5-flash')

# Planer
model = daj_mi_model()                      //Dostaje Klucz 1
lista_zadan = planer.dzialajj(model)

for zadanie in lista_zadan:                  //Pętla główna

# Developer
model = daj_mi_model()                      //Dostaje Klucz 2
kod = developer.dzialaj(zadanie, model)

# Tester
model = daj_mi_model()                      //Dostaje Klucz 3
hint = tester.dzialaj(kod, model)
# Reviewer (Dev 2)
model = daj_mi_model()                      //Dostaje Klucz 4
```

```
kod_poprawiony = reviewer.dzialaj(hint, model)
open(zadanie, "w", encoding="utf-8").write(kod_poprawiony)
```

4.3. Communication Service

Communication Service pełni rolę centralnego węzła komunikacyjnego systemu wieloagentowego. Odpowiada za koordynację wymiany informacji pomiędzy poszczególnymi agentami (GUI, Planner, Developer, Tester, TestMachine, Reviewer) przy użyciu asynchronicznej komunikacji WebSocket.

Komponent umożliwia:

- rejestrację agentów na podstawie ich ról,
- przekazywanie poleceń, kodu oraz wyników testów pomiędzy agentami zgodnie z ustalonym przepływem procesu,
- monitorowanie statusu pracy poszczególnych agentów i raportowanie postępu do interfejsu GUI,
- obsługę błędów testów i przekierowanie informacji zwrotnej do agenta Reviewer,
- zapisywanie wygenerowanego kodu oraz testów do lokalnego workspace.

Dzięki zastosowaniu `asyncio` i biblioteki `websockets`, Communication Service zapewnia nieblokującą, skalowalną i uporządkowaną komunikację pomiędzy niezależnymi komponentami systemu, pełniąc funkcję nadrzędnego koordynatora całego procesu tworzenia i testowania kodu.

4.4. Implementacja Agentów

System został zaprojektowany w architekturze wieloagentowej, w której każdy agent realizuje jasno określoną rolę w procesie generowania, testowania i weryfikacji kodu. Agenci działają jako niezależne procesy komunikujące się asynchronicznie z centralnym serwisem komunikacyjnym (HQ) za pomocą protokołu WebSocket.

Wszyscy agenci dziedziczą po klasie `BaseAgent`, która implementuje wspólną funkcjonalność:

- obsługę połączenia WebSocket z serwerem HQ,
- rejestrację agenta na podstawie jego roli,
- mechanizm odbierania i wysyłania wiadomości w formacie JSON,
- integrację z modelem generatywnym (Google Gemini).

Klasa `BaseAgent` opiera się na klasie `Endpoint`, która realizuje niskopoziomą obsługę komunikacji, w tym:

- nasłuchiwanie wiadomości przychodzących,
- delegowanie przetwarzania danych do metody `getResponse`,
- obsługę ponownego łączenia w przypadku zerwania połączenia

4.5. Test Machine

Komponent Test Machine pełni funkcję izolowanego środowiska uruchomieniowego i weryfikacyjnego w potoku produkcyjnym systemu. Jego głównym zadaniem jest fizyczne wykonanie testów jednostkowych na kodzie dostarczonym przez agenta Dev oraz walidacja wyników przed ostatecznym zapisaniem klasy w pamięci systemu.

4.5.1. Technologia i Implementacja

Serwis został zaimplementowany jako autonomiczna usługa w języku Python, wykorzystująca framework FastAPI. Wybór ten podyktowany był koniecznością zapewnienia szybkiej komunikacji asynchronicznej z *Communication Service* oraz łatwej integracji z narzędziami systemowymi.

W celu zapewnienia najwyższego poziomu bezpieczeństwa i powtarzalności testów, wdrożenie oparto na technologii konteneryzacji (Docker):

- Izolacja procesów: Każde zadanie testowe skutkuje zbudowaniem unikalnego, tymczasowego obrazu kontenera, co gwarantuje, że kod generowany przez AI nie ma dostępu do plików systemowych hosta.
- Bezpieczeństwo sieciowe: Kontenery uruchamiane są z flagą `--network none`, co całkowicie odcina testowany kod od dostępu do internetu, uniemożliwiając eksfiltrację danych.
- Zarządzanie cyklem życia: System wykorzystuje mechanizm automatycznej terminacji (`--rm`) oraz czyszczenia obrazów po każdym cyklu, co zapobiega nadmiarowemu zużyciu zasobów dyskowych infrastruktury.

4.5.2. Logika Operacyjna i Kryteria Decyzyjne

Proces weryfikacji wewnątrz Test Machine opiera się na analizie systemowego kodu wyjścia (exit code) procesu wywołującego narzędzie `pytest`. Maszyna operuje w oparciu o dwa główne scenariusze (rozwidlenia):

- Ścieżka A (Sukces): Kod wyjścia o wartości 0 oznacza pomyślne przejście wszystkich testów jednostkowych. W tym przypadku kod klasy jest uznawany za poprawny i przekazywany do etapu agregacji.
- Ścieżka B (Błąd): Kod wyjścia różny od zera (np. błąd logiczny, błąd składni lub przekroczenie czasu) inicjuje procedurę naprawczą. Test Machine przechwytuje pełny strumień logów (standard output oraz error), które są następnie przesyłane do agenta Reviewer w celu analizy i naniesienia poprawki.

Dodatkowo, serwis posiada wbudowany bezpiecznik czasowy (timeout), który chroni system przed zawieszeniem w przypadku wygenerowania przez AI nieskończonej pętli. Po przekroczeniu zdefiniowanego limitu czasowego kontener jest przymusowo zatrzymywany, a system rejestruje błąd wykonania.

4.6. Prompting

W projekcie, gdzie komunikacja odbywa się pomiędzy autonomicznymi agentami bez udziału człowieka, jakość i przewidywalność odpowiedzi generowanych przez model językowy (LLM) jest czynnikiem krytycznym. Dlatego też w warstwie inżynierii promptów przyjęto rygorystyczne zasady, których nadrzędnym celem jest uzyskanie możliwie przewidywalnych i powtarzalnych efektów pracy wszystkich agentów.

Najważniejszym założeniem projektowym była całkowita standaryzacja formatów wejściowych i wyjściowych. Aby umożliwić automatyczne przetwarzanie odpowiedzi przez kolejne moduły systemu, agenci nie mogą "rozmawiać" w sposób naturalny i swobodny. W związku z tym podjęto następujące metody standaryzacji:

Input: Prompty są konstruowane w oparciu o sztywne szablony, które wstrzykują kontekst zadania w ściśle określone miejsca. Eliminuje to niejednoznaczności interpretacyjne.

Output: Na model narzucono wymóg zwracania odpowiedzi w ustrukturyzowanej formie. Każdy agent, niezależnie od roli, zobligowany jest do umieszczania kodu w blokach Markdown lub zwracania danych w formacie JSON/tekstowym o określonej składni. Pozwala to na parsowanie kluczowych informacji i odrzucenie zbędnego tekstu.

W celu podniesienia jakości generowanego kodu i zachowania spójności działania agentów, w konstrukcji promptów zastosowano także szereg uznanych w branży technik:

Wzorzec Persony (Persona Pattern): Każdy prompt rozpoczyna się od nadania modelowi konkretnej roli i tożsamości (np. *"Jesteś ekspertem w dziedzinie testów jednostkowych w Pythonie..."*). Pozwala to na zawężenie przestrzeni poszukiwań modelu i narzucenie profesjonalnego, technicznego tonu odpowiedzi.

Ograniczenia Negatywne (Negative Constraints): Jawne instruowanie modelu, czego ma **nie robić**. Przykładowo, stosowano komendy typu *"Nie dodawaj wyjaśnień ani wstępu, zwróć wyłącznie kod"*. Jest to kluczowe dla oszczędności tokenów i łatwości parsowania.

Jasne Definiowanie Kontekstu: Prompty zawierają nie tylko polecenie, ale też niezbędny kontekst (np. fragment kodu, do którego należy napisać test), co minimalizuje ryzyko halucynacji.

Chain-of-Thought (ukryty): W bardziej złożonych zadaniach struktura promptu wymusza na modelu logiczne podejście do problemu, choć wynik końcowy jest filtrowany do samej esencji rozwiązania.

4.7. Frontend

Frontend projektu został stworzony w Next.js i odpowiada za interaktywną wizualizację komunikacji między członkami zespołu AI. Jego głównym zadaniem jest umożliwienie użytkownikowi wprowadzenia promptu, obserwowanie procesu współpracy modeli oraz odbiór finalnych plików gotowych do skopiowania.

Technologie:

- Next.js – framework React do budowy frontendowej aplikacji SSR/SPA.

- React – do budowy komponentów UI.
- TailwindCSS – do stylizacji i szybkiego tworzenia responsywnego
- WebSocket – do komunikacji w czasie rzeczywistym z backendem.

Głównym celem implementacji warstwy frontendowej jest umożliwienie użytkownikowi płynnej interakcji z autonomicznym zespołem AI za pośrednictwem czytelnego interfejsu czatu. Rozwiązanie to kładzie nacisk na wizualizację przepływu informacji pomiędzy wyspecjalizowanymi rolami, takimi jak Developer, Tester czy Koordynator, co pozwala na bieżące śledzenie postępów prac. Istotnym elementem funkcjonalnym jest przejrzysta prezentacja wygenerowanych plików z kodem, sformatowanych w sposób ułatwiający ich natychmiastowe skopiowanie, przy czym stabilność i szybkość wymiany danych w czasie rzeczywistym gwarantuje zastosowanie technologii WebSocket.

Proces działania rozpoczyna się w momencie wprowadzenia przez użytkownika promptu w aplikacji frontendowej, który następnie trafia do serwisu ClientService poprzez kanał WebSocket. ClientService inicjuje pracę, przekazując zadanie do pierwszego członka zespołu AI, a następnie każdy kolejny agent przetwarza otrzymane dane, generuje odpowiedź i przekazuje ją dalej w łańcuchu decyzyjnym. Każda z tych interakcji jest bezzwłocznie przesyłana z powrotem do frontendu i wizualizowana na czacie, dzięki czemu użytkownik widzi postęp prac "na żywo". Cały proces kończy się wygenerowaniem finalnych plików, które są wyświetlane w ostatniej wiadomości w formie gotowej do użycia.

5. Testy i Walidacja Systemu

W celu weryfikacji funkcjonalności systemu wieloagentowego przeprowadzono serię testów systemowych (end-to-end). Testy te miały na celu sprawdzenie zdolności zespołu AI do:

- Interpretacji wymagań użytkownika (Promptu).
- Doboru odpowiednich technologii (biblioteki standardowe vs zewnętrzne).
- Poprawnej implementacji logiki biznesowej oraz interfejsu.
- Autonomicznego naprawiania błędów wykrytych przez moduł *Test Machine*.

Jako przypadki testowe wybrano cztery klasyczne problemy programistyczne o zróżnicowanym poziomie skomplikowania oraz odmiennych wymaganiach środowiskowych (aplikacja konsolowa vs aplikacja okienkowa).

5.1. Scenariusz 1: Aplikacja Konsolowa (Flappy Bird)

Pierwszym testem było wygenerowanie uproszczonej wersji gry "Flappy Bird" działającej bezpośrednio w terminalu.

Cel: Weryfikacja zdolności agentów do obsługi pętli gry (game loop) i odświeżania ekranu przy użyciu wyłącznie znaków ASCII, bez zewnętrznych silników graficznych.

Przebieg: Agent *Planner* poprawnie zidentyfikował brak konieczności użycia bibliotek GUI. Agent *Developer* zaimplementował mechanizm grawitacji i wykrywania kolizji na macierzy znaków.

Rola Test Machine: Wykryto drobne błędy w imporcie modułów do obsługi czasu (time.sleep), które zostały automatycznie skorygowane w drugiej iteracji.

Wynik: Grywalna wersja konsolowa, reagująca na klawisz spacji, z poprawnym naliczaniem punktów.

5.2. Scenariusz 2: Aplikacje GUI (Pygame)

Kolejna faza testów skupiła się na zdolności systemu do integracji z zewnętrznymi bibliotekami graficznymi. Wymuszono na systemie użycie biblioteki pygame w celu stworzenia interaktywnych aplikacji okienkowych.

5.2.1. Kalkulator

Charakterystyka: Aplikacja statyczna, sterowana zdarzeniami (event-driven).

Wyzwanie: Poprawne rozrysowanie siatki przycisków oraz obsługa logiki matematycznej i stanu wyświetlacza.

Wynik: Zespół AI poprawnie oddzielił logikę obliczeń od warstwy prezentacji. *Test Machine* zweryfikowała poprawność działań matematycznych (testy jednostkowe funkcji add, subtract itd.) jeszcze przed uruchomieniem interfejsu graficznego.

5.2.2. Gra w życie (Game of Life)

Charakterystyka: Symulacja automatu komórkowego.

Wyzwanie: Implementacja algorytmu Conwaya oraz wydajne renderowanie siatki zmieniającej się w czasie rzeczywistym.

Obserwacje: W tym scenariuszu kluczowa okazała się rola agenta *Testera*. Początkowo wygenerowany kod zawierał błąd w warunkach "przeżycia" komórki. Testy jednostkowe wykryły niezgodność z regułami gry, a *Reviewer* skorygował logikę warunkową przed finalnym wyrenderowaniem okna.

5.2.3. Snake

Charakterystyka: Gra zręcznościowa czasu rzeczywistego.

Wyzwanie: Obsługa kolizji, dynamicznie rosnąca struktura danych (ogon węża) oraz sterowanie klawiaturą.

Problemy i rozwiązania: Podczas generowania tego kodu wystąpił problem z importem modułu *pygame* wewnątrz kontenera Docker (moduł *Test Machine*). System wykazał się odpornością – po otrzymaniu błędu *ModuleNotFoundError*, agent *Reviewer* zasugerował dodanie odpowiednich instrukcji do pliku *Dockerfile* lub zmianę sposobu inicjalizacji biblioteki, co pozwoliło na pomyślne uruchomienie aplikacji.

6. Wnioski

Realizacja projektu „Zespół programistyczny Agentów AI” pozwoliła na skuteczne zweryfikowanie potencjału Wielkich Modeli Językowych (LLM) w procesie automatyzacji wytwarzania oprogramowania. Główny cel, jakim było stworzenie prototypu autonomicznego systemu wieloagentowego zdolnego do analizy, implementacji i weryfikacji kodu, został osiągnięty. Przeprowadzone prace i testy pozwoliły na sformułowanie następujących wniosków kluczowych:

Skuteczność architektury Multi-Agent w porównaniu do pojedynczego modelu

Projekt potwierdził tezy stawiane w analizowanej literaturze (m.in. w *AgentCoder*), że dekompozycja procesu programistycznego na wyspecjalizowane role (Planner, Developer, Reviewer, Tester) przynosi lepsze rezultaty niż wykorzystanie pojedynczego modelu w trybie ogólnym. Specjalizacja agentów poprzez precyzyjny *Prompt Engineering* oraz nadanie im unikalnych person pozwoliło na uzyskanie kodu o wyższej spójności i mniejszej liczbie błędów logicznych.

Rola pętli sprzężenia zwrotnego (Iterative Feedback Loop)

Kluczowym elementem systemu okazał się moduł Test Machine zintegrowany z mechanizmem automatycznej naprawy kodu. Zastosowanie podejścia, w którym błędy wykryte podczas testów jednostkowych (w izolowanym środowisku Docker) są zwracane do agenta Reviewera wraz z logami błędów, pozwoliło na osiągnięcie tzw. *self-healing code*. System jest w stanie samodzielnie skorygować większość błędów składniowych i logicznych bez ingerencji człowieka, co stanowi znaczącą przewagę nad statycznym generowaniem kodu.

Bezpieczeństwo i izolacja środowiska wykonawczego

Wdrożenie konteneryzacji (Docker) dla modułu Test Machine okazało się niezbędne dla zachowania bezpieczeństwa i stabilności systemu hosta. Uruchamianie niezwyfikowanego kodu generowanego przez AI niesie ze sobą ryzyko (np. pętle nieskończone, operacje na plikach systemowych), które zostało skutecznie wyeliminowane poprzez efemeryczne kontenery z odciętym dostępem do sieci i limitami czasowymi (timeout).

Zarządzanie zasobami i ograniczenia API

Implementacja mechanizmu rotacji kluczy API (algorytm Round Robin) oraz wybór modelu Gemini (Google AI Studio) pozwoliły na efektywne zarządzanie kosztami i obejście restrykcyjnych limitów *Rate Limits*. Mimo to, głównym ograniczeniem systemu pozostaje okno kontekstowe modeli. Przy generowaniu bardziej złożonych systemów, agenci mają tendencję do "zapominania" ustaleń z wczesnych faz planowania, co wskazuje na konieczność wdrożenia w przyszłości pamięci długotrwałej (np. opartej na bazach wektorowych RAG).

Użyteczność interfejsu i transparentność procesu

Stworzenie dedykowanego Frontendu (Next.js) wizualizującego komunikację między agentami w czasie rzeczywistym (WebSocket) znacząco podniosło wartość edukacyjną i badawczą projektu. Możliwość śledzenia „tok rozumowania” zespołu AI pozwala

użytkownikowi nie tylko odebrać gotowy produkt, ale także zrozumieć proces decyzyjny stojący za konkretnymi rozwiązaniami architektonicznymi.

Podsumowując, projekt wykazał, że obecna generacja modeli LLM jest gotowa do pełnienia roli asystentów, a nawet autonomicznych wykonawców w prostych zadaniach programistycznych, pod warunkiem osadzenia ich w sztywnej, kontrolującej architekturze (Event-Driven Architecture). Dalszy rozwój systemu powinien koncentrować się na implementacji trwałej pamięci kontekstowej oraz integracji z rzeczywistymi narzędziami CI/CD.

7. Bibliografia

- [1] W.-N. J. Borzemski Leszek, „PWR Eportal - Projekt zespołowy W04IST-SM0402P,” [Online]. Available: <https://eportal.pwr.edu.pl/course/view.php?id=1218>. [Data uzyskania dostępu: 22 1 2026].
- [2] D. Huang, Q. Bu, J. Zhang, X. Xie, M. R. Lyu i H. Cui, „gentCoder: Multi-Agent-based Code Generation with Iterative Testing and Optimisation,” *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2024)*, 2024.
- [3] C. Qian, X. Cong, W. Liu, C. Yang, W. Chen, Y. Su, Y. Dang, J. Li, J. Xu, D. Li, Z. Liu i M. Sun, „Communicative Agents for Software Development,” w *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL 2024)*, 2024.
- [4] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan i S. Yao, „Reflexion: Language Agents with Verbal Reinforcement Learning,” w *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023.
- [5] u. Ye, X. Liu, Q. Wu, X. Pang, Z. Yin, L. Bai i S. Chen, „X-MAS: Towards Building Multi-Agent Systems with Heterogeneous LLMs,” *arXiv preprint arXiv:2505.16997*, 2025.
- [6] M. Fowler, „What do you mean by “Event-Driven”?,” [Online]. Available: <https://martinfowler.com/articles/201701-event-driven.html>. [Data uzyskania dostępu: 22 1 2026].